

Contexto do projeto – Suporte de TI

1. A Topologia da Rede (O Físico)

A base de tudo é como os cabos e o Wi-Fi se conectam:

- **Modem/Roteador:** Ele será o maestro da rede (Servidor DHCP). Ele vai distribuir os endereços IP para todo mundo (ex: 192.168.1.X) e fornecer o Wi-Fi para os ESP32.
- **Switch:** Conecte um cabo do Modem até o Switch. Depois, conecte o Notebook Linux e o Notebook Windows no Switch via cabo de rede. Isso garante máxima velocidade e estabilidade entre o servidor e o cliente.

2. O Notebook Linux (O Coração e o Cérebro)

O Ubuntu será o seu servidor central. Você não vai usá-lo para navegar na internet ou ler e-mails; ele ficará lá apenas rodando os serviços nos bastidores.

- **O Servidor NAS (Ferramenta: Samba / SMB):**
 - **Como funciona:** Você vai instalar um pacote chamado samba no Ubuntu. Ele permite pegar uma pasta específica do Linux (ex: /home/suporte_ti/ferramentas) e compartilhá-la na rede com um protocolo que o Windows entende nativamente.
 - **O que colocar lá:** Manuais de projetores em PDF, instaladores de antivírus, scripts de limpeza de disco e drivers de impressoras.
- **O Servidor Web (Ferramenta: Python + Flask):**
 - **Como funciona:** Como o ESP32 está configurado em Python, criar o backend (a API) em Python com o Flask é o caminho mais natural. O Flask vai rodar no Ubuntu e criar um endereço (ex: http://192.168.1.50:5000/api/chamados) que ficará "escutando" a rede 24 horas por dia, esperando o ESP32 mandar a mensagem.
 - **A Interface (Frontend):** O próprio Flask pode hospedar uma página HTML/CSS/JS onde o técnico poderá visualizar os chamados em tempo real ou onde um professor sem o ESP32 possa preencher um formulário web para pedir ajuda.
- **O Banco de Dados (Ferramenta: SQLite):**
 - **Como funciona:** O servidor Web (Flask) vai se conectar ao banco. Quando o ESP32 enviar "Problema no projetor", o Flask pega essa *string* e salva no banco de dados com a data, hora e status ("Pendente").
 - **Dica:** Para um projeto pessoais, o **SQLite** é perfeito porque é um simples arquivo local gerenciado pelo Python. Se quiser algo mais robusto, o **PostgreSQL** é mais recomendado.

3. O Notebook Windows (A Experiência Humana)

Este é o computador do técnico de TI, focado na resolução do problema. Ele não roda nenhum servidor, apenas consome o que o Ubuntu oferece.

- **Acesso ao Sistema Web:** O técnico abre o navegador no Windows e digita o IP do servidor Linux (ex: `http://192.168.1.50`). Lá ele vê o painel de chamados.
- **Acesso ao NAS:** No Explorador de Arquivos do Windows (Windows Explorer), o técnico digita `\\192.168.1.50` e a pasta de ferramentas do Linux aparece como se fosse um pen drive conectado no Windows. Ele pode mapear isso como a unidade Z: do computador.

4. O Cenário de Uso (A Jornada do Chamado)

1. **A Automação (ESP32):** O professor na sala 1 gira o seu botão rotativo, escolhe "Prob. Projetor" na tela LCD I2C e clica. O ESP32 manda a requisição via Wi-Fi para o Flask no Ubuntu.
2. **O Registro (Linux):** O Flask recebe os dados, salva no Banco de Dados e atualiza a página web.
3. **A Recepção (Windows):** O técnico, sentado em outra sala, vê o chamado pipocar na tela do navegador dele.
4. **A Resolução (NAS):** O técnico vê que é um projetor da marca X. Ele abre a pasta de rede Z: (o NAS), copia o manual técnico daquele projetor específico e vai até a sala do professor resolver a situação de forma rápida e precisa.

5. O Cenário de Uso (Medição de temperatura dos racks)

1. **A Automação (ESP32):** O dispositivo ESP32 junto ao sensor de temperatura fica próximo ao switch dentro do rack, assim que a temperatura chegar a um certo valor o ESP32 dispara um alerta.
2. **O Registro (Linux):** O Flask recebe os dados, salva no Banco de Dados e atualiza a página web.
3. **A Recepção (Windows):** O técnico, sentado em outra sala, vê o alerta pipocar na tela do navegador dele.
4. **A Resolução :** O técnico recebe o alerta crítico de temperatura e antes que danifique ou prejudique o desempenho da rede ele pode tomar uma ação preventiva.